

تمرین‌های / NumPy Optimization Practice

بهینه‌سازی در NumPy

مقدمه

این مجموعه تمرین برای یادگیری بهینه‌سازی کدهای عددی با NumPy طراحی شده است، به‌ویژه برای تبدیل حلقه‌های `for` پایتونی به عملیات برداری، `broadcasting`، `reduction` و ضرب ماتریسی. برای هر تمرین، سعی کنید ابتدا الگوی محاسبه را تشخیص دهید: آیا مسئله در اصل یک جمع روی محور خاص است؟ یک ضرب داخلی؟ یک ضرب ماتریسی؟ یک فیلتر بولی؟ یا یک `broadcast`؟

تمرین‌ها

1. افزودن یک اسکالر به همه عناصر (Add a scalar to every element)

Task: آرایه یک‌بعدی `x` با طول `n` دارید. با استفاده از حلقه، عدد 5 را به همه عناصر اضافه کنید.

```
out = np.empty_like(x)
for i in range(len(x)):
    out[i] = x[i] + 5
```

Hint: به عملیات `elementwise` بین آرایه و اسکالر فکر کنید.

2. جمع عنصر به عنصر دو بردار (Elementwise addition of two vectors)

Task: دو آرایه `a` و `b` با شکل یکسان دارید. مجموع آن‌ها را حساب کنید.

```
out = np.empty_like(a)
:for i in range(len(a))
out[i] = a[i] + b[i]
```

Hint: NumPy این نوع عملیات را مستقیماً پشتیبانی می‌کند.

3. مربع کردن همه عناصر (Square every element)

Task: برای بردار x ، برداری شامل (x_i^2) بسازید.

```
out = np.empty_like(x)
:for i in range(len(x))
out[i] = x[i] ** 2
```

Hint: یک عبارت برداری کافی است.

4. مجموع سطرهای یک ماتریس (Row sums of a matrix)

Task: برای ماتریس A با شکل (m, n) ، مجموع هر سطر را محاسبه کنید.

```
row_sums = np.zeros(A.shape[0])
:for i in range(A.shape[0])
    s = 0
    :for j in range(A.shape[1])
        s += A[i, j]
    row_sums[i] = s
```

Hint: به آرگومان `axis` در توابع reduction توجه کنید.

5. میانگین ستون‌های یک ماتریس (Column means of a matrix)

Task: میانگین هر ستون از ماتریس A را حساب کنید.

```
col_means = np.zeros(A.shape[1])
:for j in range(A.shape[1])
```

```

s = 0
:for i in range(A.shape[0])
    s += A[i, j]
col_means[j] = s / A.shape[0]

```

Hint: این هم یک reduction روی محور مناسب است.

6. ضرب ماتریس در بردار (Matrix-vector multiplication)

Task: برای ماتریس A با شکل (m, n) و بردار x با شکل (n) ، مقدار $y = Ax$ را حساب کنید.

```

y = np.zeros(A.shape[0])
:for i in range(A.shape[0])
    s = 0
:for j in range(A.shape[1])
    s += A[i, j] * x[j]
y[i] = s

```

Hint: به عملگر $@$ و تابع `np.dot` نگاه کنید.

7. مجموع حاصل ضرب‌های متناظر / ضرب داخلی (Pairwise product sum / dot product)

Task: ضرب داخلی دو بردار a و b را حساب کنید.

```

s = 0
:for i in range(len(a))
    s += a[i] * b[i]

```

Hint: هم می‌توانید `elementwise multiply + sum` انجام دهید و هم از تابع اختصاصی استفاده کنید.

8. نرمال‌سازی هر سطر (Normalize each row)

Task: ماتریس A با شکل (m, n) را طوری تبدیل کنید که هر سطر بر مجموع همان سطر تقسیم شود.

```
out = np.empty_like(A, dtype=float)
    :for i in range(A.shape[0])
        row_sum = 0
        :for j in range(A.shape[1])
            row_sum += A[i, j]
        :for j in range(A.shape[1])
            out[i, j] = A[i, j] / row_sum
```

Hint: به reduction و سپس broadcasting فکر کنید. شاید لازم باشد شکل row sums را تنظیم کنید.

9. مرکزدهی هر ستون (Center each column)

Task: از هر مقدار در ستون j ، میانگین همان ستون را کم کنید.

```
out = np.empty_like(A, dtype=float)
    :for j in range(A.shape[1])
        mean_j = 0
        :for i in range(A.shape[0])
            mean_j += A[i, j]
        mean_j /= A.shape[0]
        :for i in range(A.shape[0])
            out[i, j] = A[i, j] - mean_j
```

Hint: میانگین ستون‌ها را یک‌بار حساب کنید و بعد از broadcasting استفاده کنید.

10. فاصله اقلیدسی مربعی هر سطر تا یک بردار (Squared Euclidean distances to one vector)

Task: برای ماتریس X با شکل (m, n) و بردار c با شکل (n) ، فاصله مربعی هر سطر از X تا c را حساب کنید.

```

d = np.zeros(X.shape[0])
:for i in range(X.shape[0])
    s = 0
:for j in range(X.shape[1])
    diff = X[i, j] - c[j]
    s += diff * diff
    d[i] = s

```

Hint: c را از کل X یکجا کم کنید، سپس مربع و جمع بگیرید.

11. فاصله‌های جفتی بین همه سطرها (Pairwise distances) (between all rows)

Task: برای X با شکل (m, n)، ماتریس D با شکل (m, m) را بسازید که در آن D

i, k

(برابر فاصله مربعی بین سطر i و سطر k باشد.

```

D = np.zeros((X.shape[0], X.shape[0]))
:for i in range(X.shape[0])
:for k in range(X.shape[0])
    s = 0
:for j in range(X.shape[1])
    diff = X[i, j] - X[k, j]
    s += diff * diff
    D[i, k] = s

```

Hint: دو راه معروف دارید: broadcasting با ابعاد اضافه، یا فرمول $\|x_i - x_k\|^2 = \|x_i\|^2 + \|x_k\|^2 - 2x_i^T x_k$.

12. مجموع وزن‌دار ستون‌ها (Weighted sum of columns)

Task: برای ماتریس A با شکل (m, n) و وزن‌های w با شکل (n)، برای هر سطر مقدار y_i را به صورت $\sum_j A_{ij} w_j$ محاسبه کنید.

i, j

j

(را حساب کنید.

```

y = np.zeros(A.shape[0])
:for i in range(A.shape[0])
    s = 0
:for j in range(A.shape[1])
    s += A[i, j] * w[j]
y[i] = s

```

Hint: این دقیقاً یک ضرب ماتریس در بردار است.

13. ساخت one-hot از برچسب‌ها (One-hot encode class labels)

Task: برچسب‌های صحیح y با شکل (m) و مقادیر بین 0 تا $k-1$ دارید. یک ماتریس one-hot با شکل (m, k) بسازید.

```

Y = np.zeros((len(y), k), dtype=int)
:for i in range(len(y))
    Y[i, y[i]] = 1

```

Hint: از advanced indexing استفاده کنید.

14. جمع مقادیر بر اساس گروه (Sum values by group)

Task: آرایه $values$ با شکل (n) و آرایه $groups$ با شکل (n) دارید که مقادیرش بین 0 تا $k-1$ است. مجموع مقادیر هر گروه را حساب کنید.

```

out = np.zeros(k)
:for i in range(len(values))
    out[groups[i]] += values[i]

```

Hint: به توابعی از نوع bin counting یا تجمیع بر اساس اندیس نگاه کنید.

15. میانگین متحرک (Moving average)

Task: برای آرایه یک بعدی x و اندازه پنجره w ، میانگین متحرک را محاسبه کنید.

```
out = np.zeros(len(x) - w + 1)
for i in range(len(out)):
    s = 0
    for j in range(w):
        s += x[i + j]
    out[i] = s / w
```

Hint: به convolution، cumulative sums یا sliding window views فکر کنید.

16. شمارش عناصر بزرگ‌تر از آستانه در هر سطر (Count values exceeding a threshold per row)

Task: برای هر سطر ماتریس A ، تعداد عناصری را که از t بزرگ‌ترند بشمارید.

```
counts = np.zeros(A.shape[0], dtype=int)
for i in range(A.shape[0]):
    c = 0
    for j in range(A.shape[1]):
        if A[i, j] > t:
            c += 1
    counts[i] = c
```

Hint: مقایسه در NumPy آرایه بولی می‌سازد.

17. جایگزینی شرطی مقادیر (Replace values conditionally)

Task: آرایه x را طوری تبدیل کنید که مقادیر منفی به 0 تبدیل شوند و بقیه همان‌طور بمانند.

```
out = np.empty_like(x)
for i in range(len(x))
```

```

        :if x[i] < 0
        out[i] = 0
        :else
        out[i] = x[i]

```

Hint: از boolean masking یا `np.where` استفاده کنید.

18. فیلتر کردن با ماسک بولی (Boolean mask filtering)

Task: همه اعداد زوج را از آرایه صحیح `x` برگردانید.

```

        [] = result
        :for i in range(len(x))
        :if x[i] % 2 == 0
        result.append(x[i])
        result = np.array(result)

```

Hint: اندیس‌گذاری بولی راه طبیعی این کار است.

19. پیش‌بینی دسته‌ای در رگرسیون خطی (Batch linear regression prediction)

Task: با داشتن `X` با شکل (m, n) ، بردار وزن `w` با شکل (n) و بایاس `b`، پیش‌بینی $(y_hat = Xw + b)$ را حساب کنید.

```

y_pred = np.zeros(X.shape[0])
:for i in range(X.shape[0])
    s = 0
:for j in range(X.shape[1])
    s += X[i, j] * w[j]
y_pred[i] = s + b

```

Hint: از ضرب ماتریسی و broadcasting استفاده کنید.

20. گرادیان MSE برای رگرسیون خطی (linear regression)

Task: با داشتن X ، y ، w و b ، گرادیان تابع زیان

$$L = \frac{1}{m} \sum_i (\hat{y}_i - y_i)^2$$

را بدون حلقه حساب کنید، جایی که $(\hat{y} = Xw + b)$ است.

Hint: به residual vector و ترانهاده X فکر کنید.

21. سافت مکس سطری (Softmax by row)

Task: برای ماتریس logits به نام Z با شکل (m, k) ، softmax هر سطر را حساب کنید.

```
out = np.empty_like(Z, dtype=float)
for i in range(Z.shape[0]):
    denom = 0
    for j in range(Z.shape[1]):
        denom += np.exp(Z[i, j])
    for j in range(Z.shape[1]):
        out[i, j] = np.exp(Z[i, j]) / denom
```

Hint: برای پایداری عددی، قبل از $\exp$ کردن، بیشینه هر سطر را از همان سطر کم کنید.

22. محاسبه ماتریس کوواریانس (Compute covariance matrix)

Task: برای ماتریس داده X با شکل (m, n) که سطرها نمونه و ستون‌ها ویژگی‌اند، ماتریس کوواریانس را حساب کنید.

Hint: ابتدا ستون‌ها را center کنید، بعد از فرمول $(\frac{1}{m-1} X_c^T X_c)$ استفاده کنید.

23. یافتن اندیس بیشینه در هر سطر (Find the index of the maximum in each row)

Task: برای هر سطر از A ، اندیس ستونی بزرگترین مقدار را برگردانید.

```
idx = np.zeros(A.shape[0], dtype=int)
    :for i in range(A.shape[0])
        best_j = 0
        :for j in range(1, A.shape[1])
            :if A[i, j] > A[i, best_j]
                best_j = j
            idx[i] = best_j
```

Hint: یک تابع استاندارد برای $\arg \max$ وجود دارد.

24. ساخت Gram matrix (Build a Gram matrix)

Task: برای ماتریس X با شکل (m, n) ، ماتریس G را بسازید که در آن G

i, k

$(x_i^T x_k)$ است.

```
G = np.zeros((X.shape[0], X.shape[0]))
    :for i in range(X.shape[0])
        :for k in range(X.shape[0])
            s = 0
            :for j in range(X.shape[1])
                s += X[i, j] * X[k, j]
            G[i, k] = s
```

Hint: این مسئله را به شکل یک ضرب ماتریسی بازنویسی کنید.

25. چالش کوچک: مرحله تخصیص در Mini k-means (challenge: k-means assignment step)

Task: داده‌ها X با شکل (m, n) و centroid‌ها C با شکل (k, n) را دارید. هر نقطه را به نزدیک‌ترین centroid بر اساس فاصله اقلیدسی مربعی نسبت دهید.

Hint: هدف ساختن ماتریس فاصله‌ای با شکل (m, k) است. به broadcasting یا بسط فرمول فاصله فکر کنید.

تمرین‌های کوتاه تشخیصی

26. تبدیل این کد به NumPy:

```
total = 0
for x in arr:
    total += x
```

27. تبدیل این کد به NumPy:

```
count = 0
for x in arr:
    if x > 0:
        count += 1
```

28. تبدیل این کد به NumPy:

```
for i in range(len(arr)):
    arr[i] = arr[i] * 3 - 1
```

29. تبدیل این کد به NumPy:

```
for i in range(A.shape[0]):
    for j in range(A.shape[1]):
        B[i, j] = A[i, j] if A[i, j] > 0 else 0
```

30. تبدیل این کد به NumPy:

```
for i in range(A.shape[0]):
    mx = A[i, 0]
    for j in range(1, A.shape[1]):
        if A[i, j] > mx:
            mx = A[i, j]
    out[i] = mx
```

پیشنهاد ترتیب حل

1. عملیات elementwise: تمرین‌های 1، 2، 3، 17، 18
 2. Reductionها: تمرین‌های 4، 5، 7، 16، 23، 30
 3. Broadcasting: تمرین‌های 8، 9، 10، 21
 4. جایگزین‌های جبر خطی: تمرین‌های 6، 12، 19، 22، 24
 5. اندیس‌گذاری پیشرفته و تجمیع: تمرین‌های 13، 14
 6. الگوهای سخت‌تر برداری‌سازی: تمرین‌های 11، 15، 20، 25
-